

DATA STRUCTURES AND ALGORITHMS

STACKS AND QUEUES

Stacks and Queues are both linear data structures and defined as a collection of objects organized in a particular order. Both can access and retrieve data elements efficiently.

Stacks

- ★ Follow the Last-IN-First-OUT (LIFO) principle
- ★ Stacking item on top of the other and the one is the first to be removed from the stack.
- ★ Adding an item is referred as Push an item, use the **append()** method to push an item in the stack. If it reaches the maximum size, then stacks is overflow.
- ★ Removing an item is referred as Pop an item, use the **pop()** method to pop an item in the stack. If the stack is empty, then the stack is underflow.

Push an item in a stack

```
# declare stack(list) with no item
stack = []
# push an item in stack
stack.append("Holy")
stack.append("Angel")
stack.append("Univ")
size=len(stack)
for x in range(size-1,-1,-1): #start with the last item
    print(stack[x])
```

University
Angel
Holy

Push an item in a stack using the input()

```
# initialize stack
stack = []
# get stack size
size=int(input("Input size of stacks: "))
# push item in stack
for x in range(size):
    y=int(input("Push item in the stack [(0:d)] : ".format(x)))
    stack.append(y)
# display stack
for x in range(size-1,-1,-1):
    print(stack[x])
```

Input size of stacks: 5
Push item in the stack [0] : 12
Push item in the stack [1] : 5
Push item in the stack [2] : 10
Push item in the stack [3] : 9
Push item in the stack [4] : 4
4
9
10
5
12

Pop an item in a stack

```
stack = []
# push an item in stack
stack.append("Holy")
stack.append("Angel")
stack.append("Univ")
stack.append("HAU")
stack.append("City")

size=len(stack) # get the length of the stack
# display stack
for x in range(size-1,-1,-1):
    print(stack[x])
    print("Item to remove")
    print(stack.pop()) # remove the last item - City

print()
print("Items in stack")
size=len(stack) # get the length of the stack after pop
# display stack
for x in range(size-1,-1,-1):
    print(stack[x])
```

City
HAU
Univ
Angel
Holy

Item to remove
City

Items in stack
HAU
Univ
Angel
Holy

INFIX, PREFIX, AND POSTFIX

Infix - form with binary operators in between operands

Example: $1 * 2, (1 * 2) + 3 \rightarrow 1 * 2 + 3$

Postfix - form with binary operator after operands

Example: $1 * 2, (1 * 2) * 3 + \rightarrow 1 * 2 * 3 +$

Prefix - form with binary operators before operands

Example: $* 1 * 2, + (* 1 * 2) * 3 \rightarrow + * 1 * 2 * 3$

Operator Precedence & Associativity

lowest: +, - $\rightarrow$ left to right

Example: $1 - 2 - 3 = (1 - 2) - 3$

middle: *, / $\rightarrow$ left to right

Example: $1 / 2 / 3 = (1 / 2) / 3$

highest: ^ $\rightarrow$ right to left

Example: $1 ^ 2 ^ 3 = 1 ^ (2 ^ 3)$

★ Precedence determines order of evaluation:

Example: $1 + 2 * 3 ^ 4 \rightarrow 1 + (2 * (3 ^ 4))$

★ Parentheses can change the order of evaluation.

Example: $((1 + 2) * 3) ^ 4$

Infix Expression	Prefix Expression	Postfix Expression
$A + B$	$+ AB$	$AB +$
$A + B * C$	$+ A * BC$	$ABC * +$

Converting Infix, Postfix, and Prefix

★ set parentheses for each operator and its pair of operands

★ convert each operator separately (inside-out)

★ remove unnecessary parentheses

Given: $2 + 5 * 3$ Prefix: $+ 2 * 5 3$ Postfix: $2 5 3 +$	Given: $2 * 4 + 6 * 3 - 2$ Prefix: $- + * 2 4 * 6 3 - 2$ Postfix: $2 4 * 6 3 + 2 -$	Given: $5 / 2 * 3 + 6 - 3 * 2$ Prefix: $- + / 5 2 3 * 6 - 3 * 2$ Postfix: $5 2 / 3 * 6 + 3 2 * -$
---	---	---

Convert postfix to infix:

Given: $4 5 3 * 2 / +$ $= 4 ((5 * 3) 2 /) +$ $= 4 ((5 * 3) 2 /) +$ $= 4 (5 * 3 / 2) +$ $= 4 + 5 * 3 / 2$	Given: $4 3 + 6 - 4 * 3 2 ^ +$ $= (4 3 + 6 -) 4 * (3 2 ^) +$ $= (4 + 3 - 6 -) 4 * (3 ^ 2) +$ $= (4 + 3 - 6) * 4 (3 ^ 2) +$ $= (4 + 3 - 6) * 4 * (3 ^ 2)$	Given: $5 2 2 ^ 3 / 4 + 8 2 + -$ $= 5 (2 2 ^) 3 / 4 + (8 2 +) -$ $= 5 (2 ^ 2) 3 / 4 + (8 + 2) -$ $= 5 * (2 ^ 2) / 3 4 + (8 + 2) -$ $= 5 * 2 ^ 2 / 3 4 + (8 + 2) -$ $= 5 * 2 ^ 2 / 3 4 + (8 + 2) -$
Infix: $4 + 5 * 3 / 2$	Infix: $(4 + 3 - 6) * 4 * (3 ^ 2)$	Infix: $5 * 2 ^ 2 / 3 4 + (8 + 2) -$

Convert Prefix to infix:

Given: $+ 5 - * 6 2 / 3 4$ $= + 5 - ((6 * 2) / 3 4)$ $= + 5 (6 * 2) - (3 / 4)$ $= + 5 (6 * 2) - (3 / 4)$ $= 5 + (6 * 2) - (3 / 4)$	Given: $+ / * 6 3 2 - 5 + 2 6$ $= + / ((6 3) 2 - 5 + 2 6)$ $= + / (6 * 3) 2 - 5 (2 + 6)$ $= + (6 * 3) / 2 (5 - (2 + 6))$ $= (6 * 3) / 2 * (5 - (2 + 6))$	Given: $+ a - / * b c d e$ $= + a - ((b * c) d e)$ $= + a - ((b * c) / d) e$ $= + a - ((b * c) / d) - e$ $= a + ((b * c) / d) - e$
Infix: $5 + (6 * 2) - (3 / 4)$	Infix: $6 * 3 / 2 * (5 - (2 + 6))$	Infix: $a + b * c / d - e$

Stack Implementation

Infix to Postfix algorithm:

1. Initialize an empty stack for operators and an empty output list.
2. Scan the infix expression from left to right.
3. For each token:
 - ★ **Operand** (number/variable) → add directly to the output.
 - ★ **Operator:**
 - ☆ If the stack is empty → push the operator.
 - ☆ If the operator has higher precedence than the stack top → push it.
 - ☆ If it has lower or equal precedence → pop the stack top to the output, then push the current operator.
4. After scanning all tokens, pop all remaining operators from the stack to the output.
5. The output list is the postfix expression.

Infix: $6-3/2+5-2+6$

$632/-5+2-6+$

stack = $[-, /, +, -, +]$

postfix = $[6, 3, 2, /, -, 5, +, 2, -, 6, +]$

Infix to Prefix Algorithm:

1. Initialize an empty stack for operators and an empty output list.
2. Reverse the infix expression.
3. Scan the reversed expression from left to right.
4. For each token:
 - ★ **Operand** → add directly to the output.
 - ★ **Operator:**
 - ☆ If the stack is empty → push the operator.
 - ☆ If the operator has higher or equal precedence than the stack top → push it.
 - ☆ If it has lower precedence → pop the stack top to the output, then push the current operator.
5. After scanning all tokens, pop all remaining operators to the output.
6. Reverse the output list to get the final prefix expression.

Infix: $5/2*3+6-3*2$

$-+*/5236*32$

Reversed: $2*3-6+3*2/5$

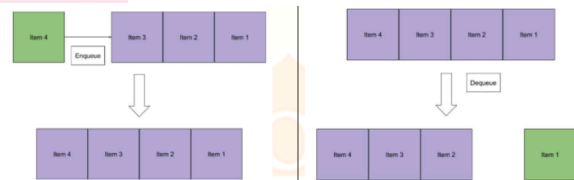
QUEUES

★ Follow the First-IN-First-OUT (FIFO) principle

★ The first one in the line is the first one to be removed in the list of items.

★ **Enqueue** - use the **append()** method to add element to the tail(end) of the queue. If it reaches the tail, then queues are overflow

★ **Dequeue** - use the **pop()** method to remove the first front item in the queue. If the queue is empty, then queues are underflow.



Dequeue item in a Queue

```
queue = []
queue.append("Holy")
queue.append("Angel")
queue.append("Univ")
queue.append("HAU")
print(queue)

# Removes the first item
print("Remove first item in the Queue")
print(queue.pop(0)) # return the value then remove the element
print(queue)
```

['Holy', 'Angel', 'Univ', 'HAU']
Remove first item in the Queue
Holy
['Angel', 'Univ', 'HAU']

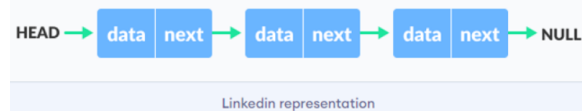
LINKED LISTS

Linked List is a very commonly used linear data structure which consists of group of nodes in a sequence.

Each **node** holds its own data and the address of the next node hence forming a chain like structure. Linked Lists are used to create trees and graphs.

A linked list is a sequence of data elements, which are connected together via links. Each data element contains a connection to another data element in a form of a pointer. A linked list is a series of connected "nodes" that contains the address of the next node.

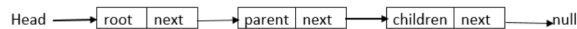
Each node can store a point which may be a number, string or any other type of data.



How node is reference in a Linked list

- ★ a data item
- ★ an address of another node

Creation of Linked Lists



First: Create a Linked lists class of Linked Lists

```
#create a class for a linked list
class LinkedList:
```

```
    def __init__(self):
        self.head = None
    if __name__ == '__main__':
        linked_list = LinkedList()
```

Second: Create a Linked Lists Node

```
class Node:
    # Creating a node
    def __init__(self, item):
        self.item = item # assigning value to a node
        self.next = None # assigning the initial value of a linked lists
```

Assign a value to a Linked Lists Node

- in assigning the value of the head node you have to include the class name
- use variables that can easily identify the node connection

```
# assigning the head of a linked list
linked_list.head = Node("root")
# assign values to nodes
second = Node("parent")
third = Node("children")
```

Connect the Linked Lists Node

```
# Connect nodes
linked_list.head.next = second # point the head node to the second node
second.next = third # point the second node to the third node
```

Display the Connected Nodes of a Linked Lists

Use a looping statement to display all the node connection of a linked list

```
# Use looping statement to print the linked list item
while linked_list.head != None:
    print(linked_list.head.item, end=" ")
    linked_list.head = linked_list.head.next
```

Output:
Jan Feb March

```
# output if no item method in the linked list
while linked_list.head != None:
    print(linked_list.head, end=" ")
    linked_list.head = linked_list.head.next
```

Output:
__main__Node object at 0x03C392F8>
__main__Node object at 0x03C39370>
__main__Node object at 0x03C393D0>

```
# output if no next method in the linked list
while linked_list.head != None:
    print(linked_list.head.item, end=" ")
    linked_list.head = linked_list.head
```

Output:
Jan Jan Jan Jan Jan Jan Jan Jan
Jan Jan Jan Jan Jan Jan Jan Jan
Jan Jan Jan Jan Jan Jan Jan Jan

Sample Linked Lists Program

Sample Program for Linked Lists of the given diagram



```
# Linked list implementation in Python
class Node:
    # Creating a node
    def __init__(self, item):
        self.item = item
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None
    if __name__ == '__main__':
        linked_list = LinkedList()
```

Output:
Jan
Feb
Mar

```
# Assign item values
linked_list.head = Node("Jan")
second = Node("Feb")
third = Node("Mar")
```

```
# Connect nodes
linked_list.head.next = second # point the head node to the second node
second.next = third # point the second node to the third node
```

```
# Print the linked list item
while linked_list.head != None:
    print(linked_list.head.item, end=" ")
    linked_list.head = linked_list.head.next
```

Insert a linked list at the beginning - the new node will be the current head of the linked list, so the current head becomes the second node and becomes the second data element.

```

class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class SLinkedList:
    def __init__(self):
        self.head = None

# Print the linked list
def listprint(self):
    printval = self.head
    while printval is not None:
        print (printval.data)
        printval = printval.next
    def AtBeginning(self, newdata):
        NewNode = Node(newdata)
        # Update the new nodes next val to existing node
        NewNode.next = self.head
        self.head = NewNode

list = SLinkedList()
list.head = Node("Jan")
L2 = Node("Feb")
L3 = Node("Mar")

list.head.next = L2
L2.next = L3

list.AtBeginning("Dec") # call the function and pass the value
list.listprint()

# You can also use the input() function to assign a new value
#X=input("Enter a month:")
#list.AtBeginning(X)

```

Output:

```

Dec
Jan
Feb
Mar

```

and the existing node after the new node is inserted.

```

class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class SLinkedList:
    def __init__(self):
        self.head = None

# Function to insert node
def Inbetween(self, middle_node, newdata):
    if middle_node is None:
        print("No Node exists")
        return
    NewNode = Node(newdata)
    NewNode.next = middle_node.next
    middle_node.next = NewNode

# Print the linked list
def listprint(self):
    printval = self.head
    while printval is not None:
        print (printval.data)
        printval = printval.next

X=input("Enter month name to insert:")
list = SLinkedList()
list.head = Node("Jan")
L2 = Node("Feb")
L3 = Node("Mar")

list.head.next = L2
L2.next = L3

list.Inbetween(list.head.next, X) # Current node is L3
list.listprint()

```

Output:

```

Enter month name to insert: Sept
Jan
Feb
Sept
Mar

```

Insert a linked list at the end - the current last node of the linked list becomes the second last data node and the new node becomes the last node.

```

class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class SLinkedList:
    def __init__(self):
        self.head = None

# Function to add newnode
def AtEnd (self, newdata):
    NewNode = Node(newdata)
    if self.head is None:
        self.head = NewNode
        return
    lastnode = self.head
    while(lastnode.next):
        lastnode = lastnode.next
    lastnode.next=NewNode

# Print the linked list
def listprint(self):
    printval = self.head
    while printval is not None:
        print (printval.data)
        printval = printval.next

Newmonth=input("Enter month name:") # input from the user
list = SLinkedList()
list.head = Node("Jan")
L2 = Node("Feb")
L3 = Node("Mar")

list.head.next = L2
L2.next = L3

list.AtEnd(Newmonth)
list.listprint()

```

Output:

```

Enter month name: Sept
Jan
Feb
Mar
Sept

```

Remove an item in linked list - remove an existing node using the key, locate the node to be deleted, then point the node to the next pointer node.

Insert item between two linked list - changing the pointer of a specific node to point to a new node, by passing in the new

```

class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None
class SLinkedList:
    def __init__(self):
        self.head = None
    def Atbegining(self, data_in):
        NewNode = Node(data_in)
        NewNode.next = self.head
        self.head = NewNode
    #Function to remove node
    def RemoveNode(self, Removekey):
        Head = self.head
        if (Head is not None):
            if (Head.data == Removekey):
                self.head = Head.next
                Head = None
                return
            while (Head is not None):
                if Head.data == Removekey:
                    break
                prev = Head
                Head = Head.next
            if (Head == None):
                return
            prev.next = Head.next
            Head = None
        def LListprint(self):
            printval = self.head
            while (printval):
                print(printval.data),
                printval = printval.next
l1 = SLinkedList()
l1.Atbegining("Jan")
l1.Atbegining("Feb")
l1.Atbegining("Mar")
l1.Atbegining("Apr")
l1.LListprint()
print("Remove Feb")
l1.RemoveNode("Feb")
l1.LListprint()

```

Output:
Apr
Mar
Feb
Jan
Remove Feb
Apr
Mar
Jan

```

class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

```

- ✧ **self.data** stores the value.
- ✧ **self.prev** stores the reference to the previous node.
- ✧ **self.next** stores the reference to the next node.

Visualizing the Links

10 ↔ 20 ↔ 30

The links are:

Node	prev	next
10	none	20
20	10	30
30	20	none

The first node has no previous node, and the last node has no next node.

Traversal in a Doubly Linked List

Forward Traversal

Start from the head and follow the next pointers.

10 → 20 → 30

```

current = head
while current:
    print(current.data)
    current = current.next

```

Output:

10
20
30

Backward Traversal

Start from the tail and follow the prev pointers.

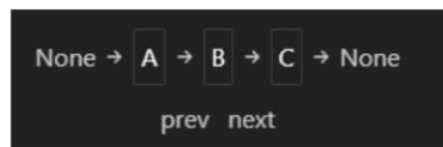
30 → 20 → 10

DOUBLY AND CIRCULAR

Doubly Linked List

A Doubly Linked List (DLL) is a linked list where each node has two pointers:

- ✧ **prev** - points to the previous node
- ✧ **next** - points to the next node



This means you can move forward and backward through the list.

Node Structure

In Python, a node for a doubly linked list may look like this:

```
current = tail
while current:
    print(current.data)
    current = current.prev
```

Output:

30
20
10

Circular Linked List

A Circular Linked List (CLL) is a linked list where the last node connects back to the first node instead of pointing to None



Notice that C points back to A

Node Structure

A circular linked list can use the same node structure as a singly linked list.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

The difference is in how the nodes are connected.

```
A.next = B
B.next = C
C.next = A
```

Why is it Called Circular?

Imagine people standing in a circle.

- ❖ A is connected to B
- ❖ B is connected to C
- ❖ C is connected back to A

There is no end.

Traversal in a Circular Linked List

Because there is no None, we must stop when we return to the starting node.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

node1 = Node(10)
node2 = Node(20)
node3 = Node(30)

node1.next = node2
node2.next = node3
node3.next = node1

head = node1

current = head

while True:
    print(current.data)
    current = current.next

    if current == head:
        break
```

Without the *if current == head* condition, the loop would continue forever.

Comparison of Linked List Types

Feature	Singly	Doubly	Circular
Pointers per node	1	2	1 (or 2 in doubly circular)
Forward traversal	Yes	Yes	Yes
Backward traversal	No	Yes	No (unless doubly circular)
Last node points to	None	None	Head node
Memory usage	Low	Higher	Low
Implementation difficulty	Easy	Medium	Medium

Real-World Examples	
Doubly Linked List	Circular Linked List
• Browser Back and Forward buttons • Undo and Redo in Microsoft Word • Music player previous/next songs	• Round-robin CPU scheduling • Multiplayer turn-based games • Circular playlists • Traffic signal control systems

RECURSION AND RECURSIVE

What is Recursion

- ✧ is a technique where a function calls itself to solve a problem.
- ✧ It's often used in computer science for problems that can be broken down into smaller, self-similar subproblems.

What is Recursive

- ✧ is a term used to describe a function, algorithm, or process that solves a problem by calling itself.
- ✧ In the context of data structures, it means using this technique to work with data that has a self-referential or nested nature.

Difference between a Recursion and Recursive

Recursion	Recursive
This refers to the concept or technique itself. It's the underlying principle in which a problem's solution relies on solutions to smaller instances of the same problem	This describes something that uses or relates to the process of recursion.
Think of it as a set of instructions	A recursive function is a function that calls itself
It is a mathematical rule	A recursive algorithm is an algorithm that uses the recursive process
is the process or method of solving a problem by breaking it down into smaller, identical subproblems	is the actual function or procedure that uses this process.

Take note:

- ✧ Use recursion to write a recursive function.
- ✧ The term recursive is then used to describe something that applies this idea. A function is called a **recursive function** because it implements the process of recursion. An algorithm is a recursive

algorithm because it follows the principles of recursion.

A recursive function must have two main parts:

- ✧ **Base Case:** The condition that stops the recursion. Without a base case, the function would call itself infinitely, leading to a stack overflow error.
- ✧ **Recursive Step:** The part where the function calls itself with a modified input to move closer to the base case.

Example : A Real-World Analogy- Imagine you have a stack of 10 boxes, and you want to open the very last box at the bottom.

- ✧ **Recursion:** The process of opening the boxes. You're repeatedly performing the same action (opening a box) on a smaller version of the problem (the remaining stack of boxes).
- ✧ **Recursive:** The instruction sheet for how to open the boxes is a recursive guide. The guide itself is what is recursive because it refers to itself.

Advantages of Recursion

- ✧ Often leads to cleaner, more elegant code for problems with a recursive nature
- ✧ Can be a more intuitive way to think about certain problems.

Disadvantages of Recursion:

- ✧ **Performance Overhead:** Each function call adds a new stack frame, which can be slower and consume more memory than a loop.
- ✧ **Stack Overflow Risk:** If the recursion is too deep. A very long linked list, for example, it can exhaust the available stack memory, causing a stack overflow error.
- ✧ Debugging can be more difficult because you have to trace the flow of many nested function calls.

Recursion in Linked Lists

- ✧ Linked lists are a great example for demonstrating recursion because their structure is inherently recursive.

- ✧ A linked list is either empty or it consists of a node followed by a smaller linked list.
- ✧ This structure makes many operations, such as traversal, searching, or insertion, elegant to implement recursively.

Traversing a Linked List Recursively The Iterative Approach (non-recursive):

```
def
print_list_iterative(head):

    current = head

    while current is not
None:

        print(current.data)

        current = current.next
```

The Recursive Approach:

```
def print_list_recursive(head):

    # Base Case: If the list is empty (head is None), stop the
recursion.

    if head is None:

        return

    # Recursive Step: Print the data of the current node.

    print(head.data)

    # Call the function again on the rest of the list
(head.next).

    print_list_recursive(head.next)
```

SEARCHING AND BASIC SORTING ALGORITHMS

Searching

- ✧ Is the process of selecting particular information from a collection of data based on specific criteria.
- ✧ In this context, we restrict the term searching to refer to the process of finding a specific item in a collection of data items.

Sequence search

- ✧ involves finding an item within a sequence using a search key to identify the specific item

Key

- ✧ a unique value used to identify the data elements of a collection.
- ✧ In collections containing simple types such as integers or reals, the values themselves are the keys.
- ✧ For collections of complex types, a specific data component has to be identified as the key.
- ✧ In many cases, a key can be as simple as a single component or attribute within the complex data type.
- ✧ In some instances, a key may consist of multiple components, which is also known as a **compound key**.

The Linear Search

- ✧ This technique iterates over the sequence, one item at a time, until the specific item is found, or all items have been examined.
- ✧ In Python, a target item can be found in a sequence using the **in** operator:

```
if key in theArray :
    print( "The key is in the array." )
else :
    print( "The key is not in the array." )
```

Searching an Unsorted Sequence

- Consider the unsorted 1-D array of integer values

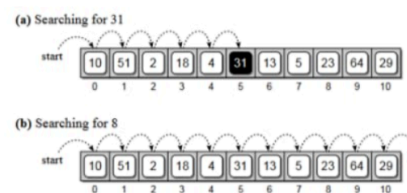


Figure 5.1: Performing a linear search on an unsorted array: (a) the target item is found and (b) the item is not in the array.

Searching on Unsorted List Elements Using in Operator:

```
mylist = [10, 51, 2, 18, 4, 31, 13, 5, 23, 64, 29]
search = 31

if search in mylist:
    print(f'{search} is in mylist')
else:
    print(f'{search} is not in mylist')
```

```
mylist = [10, 51, 2, 18, 4, 31, 13, 5, 23, 64, 29]
search = 8

if search in mylist:
    print(f'{search} is in mylist')
else:
    print(f'{search} is not in mylist')
```

Using the Linear Search Algorithm

```
def linearSearch( theValues, target ) :  
    n = len( theValues )  
    for i in range( n ) :  
        # If the target is in the ith element, return True  
        if theValues[i] == target:  
            return True  
  
    return False # If not found, return False.
```

Searching a Sorted Sequence

- ✧ A linear search can also be performed on a sorted sequence, which is a sequence containing values in a specific order.
- ✧ For example, the values in the array illustrated in Figure 5.2 are in ascending or increasing numerical order. That is, each value in the array is larger than its predecessor.

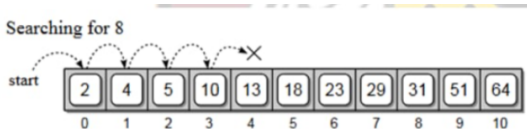


Figure 5.2: The linear search on a sorted array.

Using the linear search on a sorted sequence

```
def sortedLinearSearch( theValues, item ) :  
    n = len( theValues )  
    for i in range( n ) :  
        # If the target is found in the ith element, return True  
        if theValues[i] == item :  
            return True  
        # If target is larger than the ith element, it's not in the sequence.  
        elif theValues[i] > item :  
            return False  
  
    return False # The item is not in the sequence.
```

Sorting

- ✧ Refers to arranging data in a particular format. This algorithm specifies the way to arrange data in a particular order.
- ✧ Searching can be optimized to a very high level if the data is stored in a sorted manner.
- ✧ Also used to represent data in more readable format.
- ✧ Example: Dictionary, class list

In-place and not-in-place sorting

- ✧ **In-place sorting** - sorts the data within the original array without needing another large array. It may use a small amount of extra space for temporary values. Sorts the elements where they already are.

- ✧ **Not-in-place sorting** - requires space which is more than or equal to the elements being sorted.

- ✧ Refers to arranging data in a particular format.

Stable and not stable sorting

- ✧ **Stable sorting** - after sorting the contents and does not change the sequence of similar content in which they appear
- ✧ **Not stable sorting** - after the sorting contents, changes the sequence of similar content in which they appears.

Important terms

- ✧ **Increasing order** - if the successive element is greater than the previous one. (Ex: 2,4,6,8,9)
- ✧ **Decreasing order** - if the successive element is less than the current one. (Ex: 9,8,6,4,2)
- ✧ **Non-increasing order** - if the successive element is less than or equal to its previous element in the sequence. (Ex: 9,8,6,6,4,2)
- ✧ **Non-decreasing order** - if the successive element is greater or equal to its previous element in the sequence. (Ex: 2,4,6,6,8,9)

BUBBLE SORT

Simple sorting algorithm, it is a comparison-based algorithm with each pair of adjacent elements is compared and the elements are swapped if they are not in order.

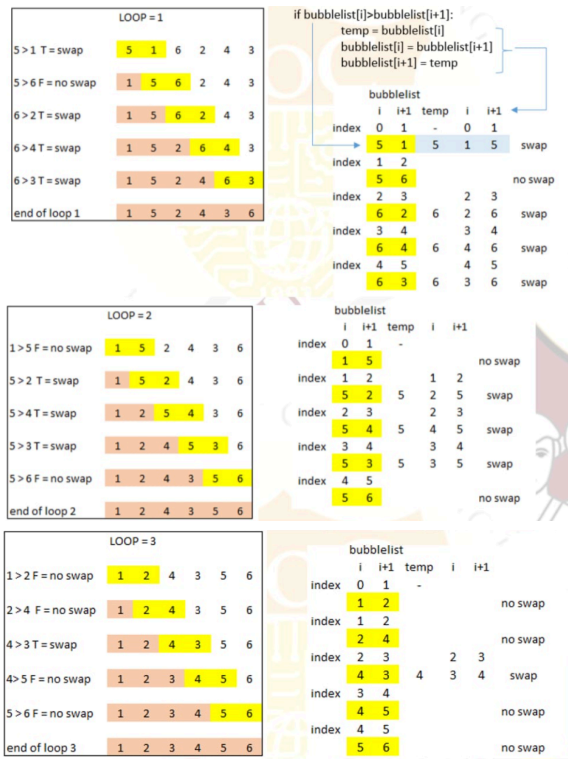
Bubble Sort Algorithm

1. Starting with the first element (index = 0), compare the current element with the next element of the array.
2. If the current element is greater than the next element of the array, swap them.
3. If the current element is less than the next element, move to the next element. Repeat Step 1.

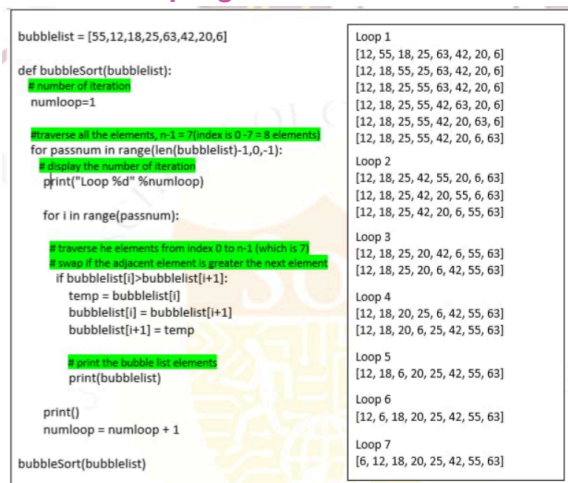
Example: Given [5,1,6,2,4,3] sort in ascending order.

Target output: 1,2,3,4,5,6.

Note: test if the element is greater than swap, otherwise retain then move to the next adjacent pair.



Bubble sort program



SELECTION SORT

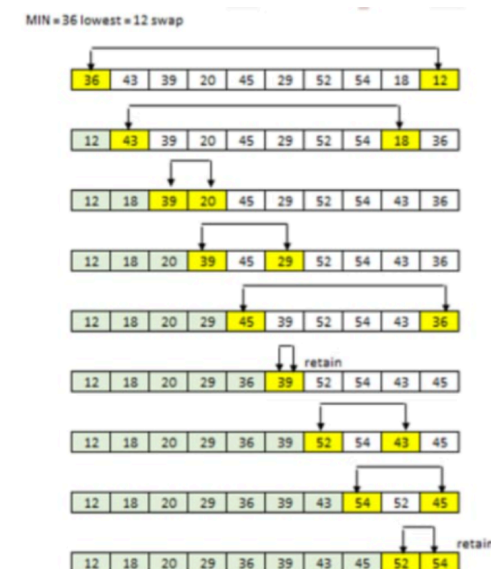
◆ Is an in-place comparison-based sorting algorithm in which the list is divided into two

parts, the sorted part at the left end and the unsorted part at the right end.

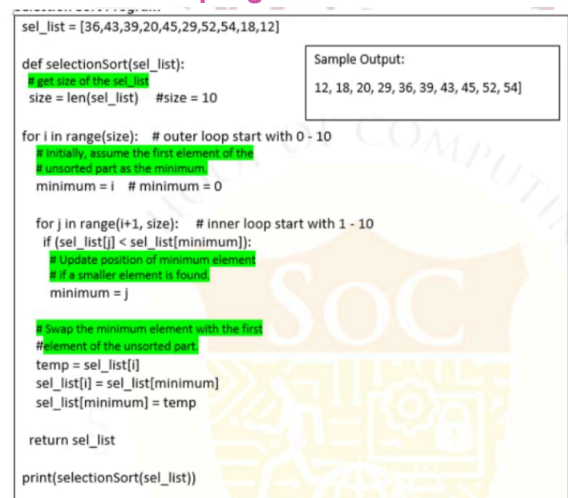
- ◆ The smallest element is selected from the unsorted array and swapped with the leftmost element and the elements becomes a part of the sorted array.
- ◆ The process continues moving unsorted array boundary by one element to the right.

Selection Sort Algorithm

1. Set MIN to location 0
2. Search the minimum element in the list
3. Swap with value at location MIN
4. Increment MIN to point to next element
5. Repeat until list is sorted



Selection sort program

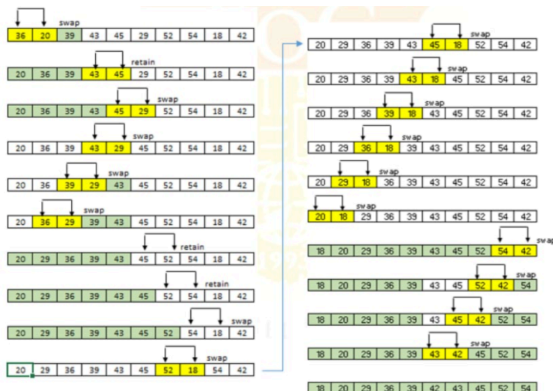


INSERTION SORT

- ◆ Is an in-place comparison-based sorting algorithm in which the sub-list is always sorted.
- ◆ An element which is to be inserted in this sorted sub-list to find its appropriate place and then it has to be inserted there.
- ◆ The element is searched sequentially and unsorted items are moved and inserted into the sorted sub-list.

Selection Sort Algorithm

1. If it is the first element, it is already sorted. return 1
2. Pick next element
3. Compare with all elements in the sorted sub-list
4. Shift all the elements in the sorted sub-list that is greater than the value to be sorted.
5. Insert the value.
6. Repeat until list is sorted



Insertion sort program

```
insert_sort = [36,43,39,20,45,29,52,54,18,42]
```

```
# Function to do insertion sort
```

```
def insertionSort(insert_sort):
```

```
    # Traverse through 1 to len(insert_sort)
```

```
    for i in range(1, len(insert_sort)):
```

```
        key = insert_sort[i]
```

```
        # Move elements of arr[0..i-1], that are  
        # greater than key, to one position ahead  
        # of their current position
```

```
        j = i-1
```

```
        while j >= 0 and key < insert_sort[j]:
```

```
            insert_sort[j+1] = insert_sort[j]
```

```
            j -= 1
```

```
        insert_sort[j+1] = key
```

```
insertionSort(insert_sort)
```

```
print("Sorted array is:")
```

```
for i in range(len(insert_sort)):
```

```
    print("%d" %insert_sort[i])
```

```
print(selectionSort(sel_list))
```

Sorted array is:

18

20

29

36

39

42

43

45

52

54

